

1 Solutions of Problems in Exercise Session 3

Exercise 1. You run the Q -learning and SARSA algorithms with zero-initialization, parameters $\varepsilon = 0.1, \alpha = 0.5$, and $\gamma = 1$ for 2 episodes, observing the following samples: $(s_t, a_t, r_t, \dots)$: $(6, \uparrow, -1, 3, \rightarrow, -1, 4, \downarrow, -1000, 7)$, $(6, \uparrow, -1, 3, \rightarrow, -1, 4, \downarrow, -1000, 7)$. Write down the Q -values for both algorithms after these two episodes.

Solution. Let's start with SARSA. The SARSA updates are defined as:

$$Q^{(t+1)}(s, a) = Q^{(t)}(s, a) + \mathbf{1}_{(s_t, a_t)=(s, a)} \alpha (r_t + \gamma Q^{(t)}(s_{t+1}, a_{t+1}) - Q^{(t)}(s_t, a_t))$$

With zero-initialization, the updates simplify as follows:

$$Q^{(1)}(s_1 = 6, a_1 = \uparrow) = \alpha(r_1 + \gamma Q^{(0)}(s_2 = 3, a_2 = \rightarrow)) = \alpha r_1 = -0.5,$$

while all other entries remain zero. The next update is $Q^{(2)}(s_2 = 3, a_2 = \rightarrow) \leftarrow \alpha r_2 = -0.5$ using the same approach. Next we have:

$$Q^{(3)}(s_3 = 4, a_3 = \downarrow) = \alpha(r_3 + \gamma Q^{(2)}(s_4 = 7, a)) = \alpha r_3 = -500.$$

Note that no action is taken in state 7 (corresponding to the hole), since after getting to this state we restart the episode - you can think of this as $Q(7, a) = 0$ for any action a . Next we are going to update the Q -values using samples from the second episode. Since some Q -estimates are now non-zero, we are going to use the full expression for updates:

$$\begin{aligned} Q^{(4)}(s_5 = 6, a_5 = \uparrow) &= Q^{(3)}(6, \uparrow) + \alpha(r_5 + \gamma Q^{(3)}(s_6 = 3, a_6 = \rightarrow) - Q^{(3)}(6, \uparrow)) \\ &= -0.5 + 0.5(-1 + 1(-0.5) - (-0.5)) = -1 \end{aligned}$$

$$\begin{aligned} Q^{(5)}(s_6 = 3, a_6 = \rightarrow) &= Q^{(4)}(3, \rightarrow) + \alpha(r_6 + \gamma Q^{(4)}(s_7 = 4, a_7 = \downarrow) - Q^{(4)}(3, \rightarrow)) \\ &= -0.5 + 0.5(-1 + 1(-500) - (-0.5)) = -250.75 \end{aligned}$$

$$\begin{aligned} Q^{(6)}(s_7 = 4, a_7 = \downarrow) &= Q^{(5)}(4, \downarrow) + \alpha(r_7 + \gamma Q^{(5)}(s_8 = 7, a) - Q^{(5)}(4, \downarrow)) \\ &= -500 + 0.5(-1000 - (-500)) = -750 \end{aligned}$$

So, the final estimate obtained by SARSA is $Q_{\text{SARSA}}(6, \uparrow) = -1, Q_{\text{SARSA}}(3, \rightarrow) = -250.75, Q_{\text{SARSA}}(4, \downarrow) = -750$, and estimates for all other state-action pairs are zero.

Now, we consider Q -learning and recall that the Q -learning updates are given by:

$$Q^{(t+1)}(s, a) = Q^{(t)}(s, a) + \mathbf{1}_{(s_t, a_t)=(s, a)} \alpha (r_t + \gamma \max_{b \in \mathcal{A}} Q^{(t)}(s_{t+1}, b) - Q^{(t)}(s_t, a_t)).$$

Observe that Q -learning performs identical updates during the first episode. Thus, we obtain $Q^{(3)}(6, \uparrow) = -0.5, Q^{(3)}(3, \rightarrow) = -0.5$ and $Q^{(3)}(4, \downarrow) = -500$. However, the updates made by Q -learning differ from those of SARSA. At the beginning of the second episode we have:

$$Q^{(4)}(s_5 = 6, a_5 = \uparrow) = Q^{(3)}(6, \uparrow) + \alpha(r_5 + \underbrace{\gamma \max_{b \in \mathcal{A}} Q^{(3)}(s_6 = 3, b)}_{=0} - Q^{(3)}(6, \uparrow)) = -0.75.$$

Note that while $s_6 = 3, a_6 = \rightarrow$ and $Q^{(3)}(s_6 = 3, a_6 = \rightarrow) = -0.5$, in Q -learning, the target is the Q -value of the best possible action, and we thus use the Q -estimate of any of the three remaining actions in s_6 with 0-values. This is even more evident in the next update when we set: $Q^{(5)}(s_6 = 3, a_6 = \rightarrow) = -0.75$ instead of -250.75 from SARSA. Finally, we update $Q^{(6)}(s_7 = 4, a_7 = \downarrow) = -750$. Thus, the final Q -learning estimates are $Q_{Q\text{-learning}}(6, \uparrow) = -0.75, Q_{Q\text{-learning}}(3, \rightarrow) = -0.75, Q_{Q\text{-learning}}(4, \downarrow) = -750$, and all other state-action pairs have Q -estimates equal to zero.

Exercise 2. You got tired of calculating Q -values by hand and have implemented Q -learning and SARSA. However, you forgot to write the update rules in the script `safeSARSA.py`. Complete the missing code, and run the script for 10,000 episodes. Examine the learned Q -values and try to explain the difference in Q -estimates for state 3. Explain why SARSA picks the safer, longer path, while Q -learning chooses the optimal, but riskier path.

Solution. After running the script, you should observe that the two methods converge to the following policies:

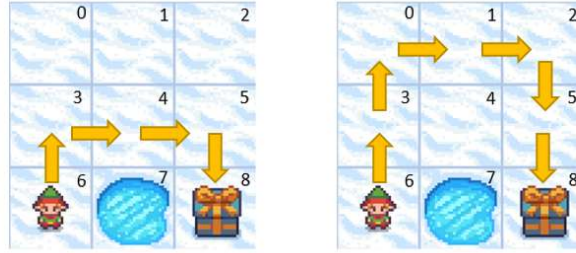


Figure 1: Learned policies using Q -learning (left) and SARSA (right).

While Q -learning converges to the optimal policy that finds the shortest path, policy found by SARSA is not optimal. From the paths shown in the figure, it is clear that the difference arises in state $s = 3$. Examining the learned Q -values, we see that for Q -learning, $Q_{Q\text{-learning}}(s = 3, a) \approx [-4, -5, -3, -5]$, where the entries correspond to the actions $\leftarrow, \downarrow, \rightarrow$, and $\uparrow$, respectively. On the other hand, for SARSA, $Q_{\text{SARSA}}(s = 3, a) \approx [-32, -17, -16, -5]$. Thus, the optimal action for Q -learning is the third action (right), while for SARSA, it is the fourth action (up).

The reason for this difference is as follows. Recall that Q -learning updates $Q(s_t, a_t)$ based on $\max_{b \in \mathcal{A}} Q(s_{t+1}, b)$, which selects the best possible action in the next state. In contrast, SARSA updates $Q(s_t, a_t)$ based on $Q(s_{t+1}, a_{t+1})$, which depends on the actual action taken in the next state.

In our example, whenever the agent was in state 4, it has always taken the action $\downarrow$ and fell into a hole, receiving a large negative reward. As a result, $Q(s = 4, a = \downarrow)$ becomes highly negative. Since SARSA updates depend on the actual next state-action pair, when transitioning from state $s = 3$ to $s = 4$ with the action down, the value of $Q(3, \rightarrow)$ is heavily penalized.

In contrast, Q -learning considers the best possible action in state $s = 4$, which corresponds to actions $\leftarrow, \uparrow$ or $\rightarrow$, all of which have Q -values closer to zero. This prevents $Q(s = 3, a = \rightarrow)$ from being updated as drastically.

Because of this difference, even though both methods are guaranteed to converge to the optimal policy with suitably decreasing ε and α , they may converge to different policies in practice.

Exercise 3. You choose daily between two restaurants, X and Y. Each day you observe independent rating $x_i, y_i \in \{1, 2, 3, 4, 5\}$, which are unbiased samples of their true mean qualities μ_X and μ_Y . You always visit the restaurant with the higher observed rating $z_i = \max\{x_i, y_i\}$. After N visits, your estimated quality of the better restaurant becomes $\frac{1}{N} \sum_{i=1}^N z_i \rightarrow \mathbb{E}z_i$. Given that the ratings follow these distributions:

$$\mathbb{P}(x_i = k) = \frac{1}{5} \text{ for } k = 1, 2, 3, 4, 5,$$

$$\mathbb{P}(y_i = k) = \frac{1}{9} \text{ for } k = 1, 2, 3, \text{ and } \mathbb{P}(y_i = k) = \frac{1}{3} \text{ for } k = 4, 5,$$

show that this procedure likely overestimates the quality of best restaurant by showing:

$$\max\{\mu_X, \mu_Y\} = \max\{\mathbb{E}x_i, \mathbb{E}y_i\} \leq \mathbb{E}\max\{x_i, y_i\} = \mathbb{E}z_i$$

Solution. First, compute the expectations of the ratings:

$$\begin{aligned} \mu_X = \mathbb{E}[x_i] &= \sum_{k=1}^5 k \mathbb{P}(x_i = k) = \sum_{k=1}^5 k \frac{1}{5} = 3 \\ \mu_Y = \mathbb{E}[y_i] &= \sum_{k=1}^5 k \mathbb{P}(y_i = k) = \sum_{k=1}^3 k \frac{1}{9} + \sum_{k=4}^5 k \frac{1}{3} = 3.67. \end{aligned}$$

Thus, the higher average rating belongs to restaurant Y, with $\max\{\mu_X, \mu_Y\} = 3.67$.

Next, compute $\mathbb{E}z_i = \mathbb{E}\max\{x_i, y_i\}$. To do this, note the following cases for z_i :

$$z_i = \begin{cases} 1, & \text{if } x_i = 1 \text{ and } y_i = 1 \\ 2, & \text{if } (x_i = 2 \text{ and } y_i < 2) \text{ or } (x_i \leq 2 \text{ and } y_i = 2) \\ 3, & \text{if } (x_i = 3 \text{ and } y_i < 3) \text{ or } (x_i \leq 3 \text{ and } y_i = 3) \\ 4 \text{ and } 5 & \text{follow similarly.} \end{cases}$$

Using the independence of x_i and y_i , we have: $\mathbb{P}(z_i = 1) = \mathbb{P}(x_i = 1)\mathbb{P}(y_i = 1) = \frac{1}{9} \frac{1}{5} = \frac{1}{45}$ and similarly $\mathbb{P}(z_i = 2) = \frac{3}{45}$, $\mathbb{P}(z_i = 3) = \frac{5}{45}$, $\mathbb{P}(z_i = 4) = \frac{15}{45}$, $\mathbb{P}(z_i = 5) = \frac{21}{45}$. Finally, compute the expected value of z_i :

$$\mathbb{E}z_i = \sum_{k=1}^5 k \mathbb{P}(z_i = k) = \frac{187}{45} \approx 4.16.$$

Since $4.16 > 3.67 = \max\{\mu_X, \mu_Y\}$, the inequality is satisfied, as required.

Exercise 4. Let s_{t+1} be a **fixed** state. Suppose both estimators are unbiased for every action a : $\mathbb{E}[Q_1^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q_2^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q^{(t)}(s_{t+1}, a)]$. Let $a^* \in \operatorname{argmax}_b Q_1^{(t)}(s_{t+1}, b)$ be the action that looks optimal under the first estimator. Show that its value under the second estimator satisfies:

$$\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*)] \leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)].$$

Solution. Let $\mathcal{M}$ denote the set of all true optimal actions, i.e., $\mathcal{M} = \{a : Q^{(t)}(s_{t+1}, a) = \max_b Q^{(t)}(s_{t+1}, b)\}$. By the law of total expectation, we can write:

$$\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*)] = \mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \in \mathcal{M}] \mathbb{P}(a^* \in \mathcal{M}) + \mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \notin \mathcal{M}] \mathbb{P}(a^* \notin \mathcal{M})$$

Recall that we assumed that the estimates $Q_1^{(t)}$ and $Q_2^{(t)}$ are unbiased, meaning that $\mathbb{E}[Q_1^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q_2^{(t)}(s_{t+1}, a)] = \mathbb{E}[Q^{(t)}(s_{t+1}, a)]$ for all actions a . Thus, in particular,

for action $a = a^*$, we have:

$$\begin{aligned}\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \in \mathcal{M}] &= \mathbb{E}[Q_1^{(t)}(s_{t+1}, a^*) | a^* \in \mathcal{M}] \\ &= \mathbb{E}[Q^{(t)}(s_{t+1}, a^*) | a^* \in \mathcal{M}] = \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)].\end{aligned}$$

On the other hand, when $a^* \notin \mathcal{M}$:

$$\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \notin \mathcal{M}] = \mathbb{E}[Q^{(t)}(s_{t+1}, a^*) | a^* \notin \mathcal{M}] \leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)],$$

since $a^* \notin \mathcal{M}$ implies that there exists another action with higher Q -value.

Combining these two equations, we get:

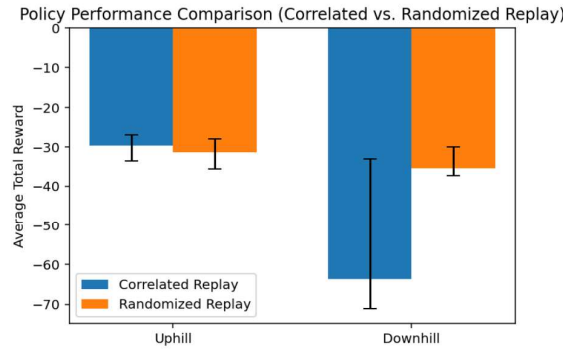
$$\begin{aligned}\mathbb{E}[Q^{(t)}(s_{t+1}, a^*)] &= \underbrace{\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \in \mathcal{M}] \mathbb{P}(a^* \in \mathcal{M})}_{\leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)]} + \underbrace{\mathbb{E}[Q_2^{(t)}(s_{t+1}, a^*) | a^* \notin \mathcal{M}] \mathbb{P}(a^* \notin \mathcal{M})}_{\leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)]} \\ &\leq \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)] (\mathbb{P}(a^* \in \mathcal{M}) + \mathbb{P}(a^* \notin \mathcal{M})) = \max_a \mathbb{E}[Q^{(t)}(s_{t+1}, a)].\end{aligned}$$

Finally, equality occurs only when a^* is guaranteed to belong to $\mathcal{M}$, i.e., when $\mathbb{P}(a^* \notin \mathcal{M}) = 0$. Since a^* is random, this means that usually the inequality will have some slack, meaning the estimator underestimates the Q -values. $\square$

Exercise 5. A company developing autonomous driving systems has encountered failures in its DQN training pipeline after removing the replay buffer: vehicles accelerate uncontrollably on downhill segments.

Inspect the script `downhill.py`, implement a replay buffer, and evaluate whether it resolves the issue. Explain how the replay buffer affects training stability.

Solution. The implemented algorithm switches between the uphill and downhill environments every 2000 steps. When using the correlated method (without experience replay buffer), the last 10 samples are taken and used for updates. In contrast, with the experience replay buffer, 10 samples are randomly selected from the buffer. After training, the policies are tested on both uphill and downhill environments.



Upon examining the results, we see that the randomized replay method performs better for downhill environment. This is due to the spurious correlation between the samples used for training in the correlated model. In general, we want to train our models on as diverse a set of experiences as possible. This includes even experiences that may have occurred some time ago. For instance, agent should not forget how to brake when going downhill just because it has been going uphill lately.